NATIONAL UNIVERSITY OF SINGAPORE
SCHOOL OF COMPUTING

FINAL EXAM FOR
Semester 2, AY2013/2014

CS3230 – DESIGN AND ANALYSIS OF ALGORITHMS

26 April 2014 Time Allowed: 2 hours

Instructions to Candidates:

1. This examination paper contains **FIVE** (5) questions and comprises **TEN** (**10**) printed pages, including this page.
2. Answer **ALL** questions.
3. Write ALL your answers in this examination book.
4. This is an **OPEN BOOK** examination.
5. You can **freely quote** standard algorithms and data structures covered in the lectures, textbook, homework, and programming assignments. However, please explain **any modifications** you make to them.
6. Unless otherwise specified, you are expected to **prove (justify)** your results.
7. All logarithms are in base 2, unless otherwise stated.

Matriculation Number: _____

QUESTION	POSSIBLE	SCORE
Q1	10	
Q2	10	
Q3	10	
Q4	5	
Q5	5	
TOTAL	**40**	

Score: (__+__) + (__) = (___)

Q1. (10 points)

(a). (4 points) Solve the following recurrence, giving your answer for Q(n) in Θ-notation:

$Q(n) = Q(n-1) + 5n^2$ for $n > 1$, $Q(1) = 5$.

Solution: *$Q(n) = \Theta(n^3)$*

$R(n) = 6R(n/2) + \Theta(n^2)$ for $n > 1$, $R(1) = \Theta(1)$.

Solution: *$R(n) = \Theta(n^p)$, where $p = \log_2 6$. (Case 1 of Master Theorem)*
Students do not need to use calculator to compute $\log_2 6 = 2.584$ but are expected to aware that $\log_2 6$ is greater than $\log_2 4 = 2$, so that Case 1 of Master Theorem is applicable.

(b). (6 points) Suppose that instead of expanding a table by doubling its size when the table is full, we expand it by multiplying its size by 4 when the table is full. Show that the amortized cost of TABLE-INSERT using this strategy is bounded above by a constant. What is the constant amortized cost per operation? Use any method that you like.

Solution:
Using the aggregate method
Cost of ith operation is $c_i = 1$ *when* $(i-1) \neq 4^k$
$= 1 + 4^k$ *when* $(i-1) = 4^k$
Total cost of n operations is $(c_1 + c_2 + \ldots + + c_n)$
$= (1 + 1 + 1 \ldots + 1) + (4^0 + 4^1 + 4^2 + \ldots + 4^R)$ *where* $R = \lfloor \lg_4 n \rfloor$
$\leq n + (4^{R+1} - 1)/(4-1) \leq n(1 + 4/3)$
Amortized cost = 2.3333n

Using Accounting Method:
Each operation is charge \$2.3333.
** \$1 used up insert the item into the table.*
** \$1 saved up to move the item into expanded table.*
** \$0.333 save up to help pay to move 1/3 item into expanded table.*

Now that just after table expansion, from N to 4N,
there are N items (with no \$\$)
and 3N empty slots, saving up 3N\$1.333 = \$4N before next expansion.*

Expected score for this question 1 is (2+1) + 5 = 8 points

Q2. (TREASURE-HUNT) (10 points)

There are **N** treasure boxes in an island left by some rich king of the past. Each treasure box is of the same value (1 kg of pure gold). However, this island is full of traps. If you take a treasure box, some other treasure boxes cannot be taken. Otherwise a bomb will destroy the island (including yourself).

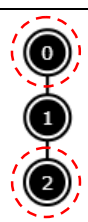
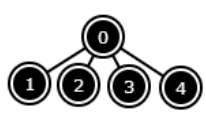
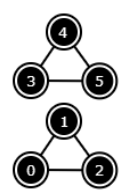
Fortunately, you somehow get a hand of the king's secret note that records the structure of the treasure boxes. The king's secret note first starts with two integers **N** and **M** in one line followed with a blank line. The king has numbered the treasure boxes from [0..**N**-1]. Each of the next **M (0 ≤ M ≤ N*(N-1)/2)** lines in the king's secret note contain two different integers **i** and **j** which means that the king has set up a trap such that if **both** treasure boxes **i** and **j** are taken away from their position, the island will self-destruct. ~~The integer **i** and **j** also imply that there is a straight path in the island that connects the location of treasure box **i** and the location of treasure box **j**~~ (note: this sentence causes minor ambiguity).

The **optimization version** of **TREASURE-HUNT** is this: Given a king's secret note, what is the maximum number of treasure boxes that you can take out from that island **alive**?

The **decision version** of **TREASURE-HUNT** called **TREASURE-HUNT-DEC** is this: Given a king's secret note, can we take out **K** out of **N** treasure boxes from that island **alive**?

(a). (2 points) Understanding TREASURE-HUNT

Given these five different king's secret notes, please list down the treasure boxes that you will take out from the island to solve the optimization version of TREASURE-HUNT (note: any correct answer is OK). The first king's secret note has been solved as an example.

Secret Note 1	Secret Note 2	Secret Note 3	Secret Note 4	Secret Note 5
3 2 0 1 1 2	2 1 0 1	5 4 0 1 0 2 0 3 0 4	5 8 0 1 0 2 0 3 0 4 1 2 1 4 2 3 3 4	6 6 0 1 0 2 1 2 3 4 3 5 4 5
				
Ans: 2 boxes Take {0, 2}	Ans:__box(es) Take {_____}	Ans:__box(es) Take {_____}	Ans:__box(es) Take {_____}	Ans:__box(es) Take {_____}
	Ans: 1 box(es) Take {0/1}	Ans: 4 box(es) Take {1, 2, 3, 4}	Ans: 2 box(es) Take {1+3/2+4}	Ans: 2 box(es) 9 possibilities

(b). (2 points) Prove that TREASURE-HUNT-DEC is in NP

Hint: Let the certificate be the set of treasure boxes that you take and a target integer K.

Solution: TREASURE-HUNT-DEC is actually an INDEPENDENT-SET problem. The certificate has been fixed to be the set of treasure boxes that you take and a target integer K to simplify grading. The verifier algorithm will check in O(N) if all treasure boxes that are taken are numbered between [0..N-1] (the treasure box numbers in King's secret note) and whether we indeed take ≥ K treasure boxes. Then, the verifier algorithm will check in O(M): for any treasure box pair i and j in the king's secret note, we cannot have both treasure box i and j taken. Notice that all these can be done in polynomial time, so we have shown that TREASURE-HUNT-DEC is in NP.

(c). (5 points) Prove that 3-CNF-SAT $\leq_p$ TREASURE-HUNT-DEC.

For this question, you can assume that 3-CNF-SAT has been proven to be NP-Complete.

Formal definition of 3-CNF-SAT problem:
Input: A 3-CNF formula ϕ (satisfiability instance in Conjunctive Normal Form and with exactly 3 distinct literals per clause)
Output: Whether the 3-CNF ϕ is satisfiable?(note: there is a single letter typo)

Please follow the steps below:

1. Write down the general idea of your polynomial-time reduction algorithm that takes in a 3-CNF formula ϕ and convert it into an instance of TREASURE-HUNT-DEC **(1 point)**

*Hint: Consider a triangle with 3 vertices (see the king's secret note 5 in **Q2 (a)**).*

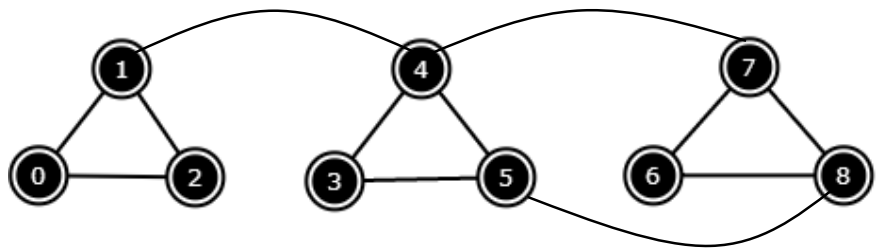
*Solution: For each 3-CNF clause with three literals, we create a triangle graph. Inside this triangle graph, we can only pick one vertex to be included in the Independent Set. This key step is hopefully aided by the hint so that more students can get the answers rather than leaving this part blank. Next, we also add an edge between a variable in triangle **i** to the negation of that variable in another triangle **j**. This will cause the assignment to be consistent (such edge constraint has been shown when we discuss the reduction from 3-CNF-SAT to CLIQUE in class so hopefully more than half of the class got this second condition right). Now, if the 3-CNF formula with N clauses is satisfiable, then the graph will have Independent Set that contains N vertices.*

Reference: Sanjoy Dasgupta et al's book: "Algorithms" page 249-250.

Expected score for this question 2 is (1.5)+(1.5)+(0.5+0.5+0.5+0.5)+(1) = 6 points

2. Apply your reduction algorithm on the following 3-CNF formula ϕ
   $\phi = (x_1 \vee x_2 \vee \neg x_4) \wedge (x_1 \vee \neg x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \neg x_3)$
   to produce an instance of TREASURE-HUNT-DEC **(1 point)**.

Solution:
Vertex 0, 3, 6 are for x_1, Vertex 1, 4, 7 are for x_2, $\neg x_2$, and x_2
Vertex 2 is for $\neg x_4$; Vertex 5 and 8 are for x_3 and $\neg x_3$.
The graph is shown below and we are looking for Independent Set of size 3 (because the 3-CNF formula ϕ has 3 clauses).



3. Prove the correctness of your reduction algorithm **(2 points)**

Solution (other valid arguments are accepted):

If the 3-CNF-SAT ϕ with N clauses is satisfiable → the graph has an Independent Set S of size N vertices.

The satisfying assignment must have set at least one literal in each clause to be true, we pick the corresponding vertex in the graph to be part of the Independent Set. Because of the way we construct the graph, there must be at least one vertex out of each triangle that can be chosen and these chosen vertices must be independent with respect to other triangles (other clauses) as we will not have a variable and its negation in a satisfying assignment. Thus the Independent Set must have size N vertices.

If the 3-CNF-SAT ϕ with N clauses is satisfiable ← the graph has an independent set S of size N vertices.

For any variable x, the set S cannot contain vertices labeled both x and $\neg x$ as such pair are connected with an edge in our graph. Since S has N vertices, it must have one vertex that represents a variable x (or its negation) that is set to true in each triangle. We set the corresponding variable as true in each clause, thus the 3-CNF-SAT ϕ must be satisfiable.

4. Analyze the time complexity of your reduction algorithm. Is it polynomial? **(1 point)**

Solution:

We create the same number of vertices as with the number of the input literals.
*We create polynomial number of edges, which is 3*Clauses (for the triangles) + at most Clauses^2 number of additional edges for preventing conflict.*
Thus the reduction is a polynomial time reduction.

(d). (1 point) Conclusion

Due to the results of **Q2 (b)** & **(c)**, what can you conclude about TREASURE-HUNT-DEC?

Solution: TREASURE-HUNT-DEC is NP-Complete (the essence of Lecture7+8 of CS3230).

Q3. (Special case of TREASURE-HUNT) (10 points)

(a). (3 points) Trying your luck…

Before you actually take out any treasure box (and risk blowing out yourself), you decide to explore the island first. You walk from the location of one treasure box to another treasure box in that island. To your amazement, you find out that you are always able to go from any treasure box **i** to any treasure box **j** via only one unique path where **i, j ∈ [0..N-1]**. King secret's note 1, 2, and 3 in **Q2 (a)** above exhibits this property.

(a.1). (1 point) From these observations, you conclude that the connection of these treasure boxes in the island is actually a *special* graph structure called a

(circle one: Directed Acyclic Graph/Tree/Bipartite Graph).

Solution: the most precise answer is a tree, a connected graph with one unique path between any pair of vertices. Note that all trees are bipartite graphs, so we will also give marks if students circle bipartite graph. However doing so will complicate the thinking process required for part b. Note that it is incorrect to select both options as you can only circle one.

(a.2). (1 point) You also conclude that if you run Depth First Search (DFS) algorithm on the graph that represents the connection of these treasure boxes from any starting treasure box, you will **only** encounter

(circle one: tree/back/forward/cross) edge(s).

Solution: as a tree is acyclic, we will not see back edge and all edges are DFS tree edges.

(a.3). (1 point) You also conclude that if you run Breadth First Search (BFS) algorithm on the graph that represents the connection of these treasure boxes, you will get

(circle one: the same/different) spanning tree

compared to the DFS spanning tree. Please assume that BFS/DFS starts from treasure box 0.

Solution: in a tree, the DFS and BFS spanning tree starting from the same vertex is the same.

(b). (7 points) You are feeling lucky!

Assuming that the property that you have observed in **Q3 (a.1)** is true, design a **Dynamic Programming** algorithm for solving this special case of TREASURE-HUNT that runs in **O(N)** time and uses at most **O(N)** space to compute.

Design a recursive formula that solves this special case of TREASURE-HUNT.

Hint: For each treasure box, you have two possible actions: Take or not take it. Start your Dynamic Programming algorithm by deciding to take or not to take treasure box 0 and then decide what you should do with the remaining treasure boxes (the subproblems).

(b.1). (2 points) List down all the recursive cases.

Solution: Max Independent Set on tree (unweighted version). Similar solution for the weighted version (can easily be converted to unweighted version) is in page 368 of CP3.

Let MIS(u, t) be the size of the Max Independent Set of a vertex rooted at u and we decide to take (t = true) or not take vertex u (t = false).

MIS(u, true) = 1 + ∑MIS(v, false), for all children v of u
(children of u cannot be taken in order not to violate the Independent Set property).

MIS(u, false) = ∑max(MIS(v, false), MIS(v, true)), for all children v of u
(one of these two actions is the maximum, we can choose either option as it will not violate the Independent Set property).

Note: 'Children' is an Adjacency List that contains the directed version of the tree (parent points to its children; but the children do not point to parents). This is doable by doing just one DFS traversal starting from vertex 0 and fix the direction of DFS tree edges.

Note: Alternative formulation that uses the notion of children and grandchildren exists (see Sanjoy Dasgupta et al's book, "Algorithms", page 175-177).

(b.2). (1 point) List down all the base cases.

Solution:

MIS(u, true) = 1 if u is a leaf, MIS(u, false) = 0 if u is a leaf

(b.3). (2 points) Show an instance of overlapping subproblems in the recursive formula that you have written down in **Q3 (b.1)** when applied to the king's secret note 3 as in **Q2 (a)**. (note: during exam, a few (< 5) of you encounter issues with this small test case and request if the condition to use king's secret note 3 is relaxed, see the remarks below).

Solution:

We fix the reference point to king's secret note 3 as in Q2 (a) for easier grading although for other recursive formulation, i.e. the children-grandchildren formulation, it may be harder to show overlapping subproblems using this small instance and students have to draw a slightly larger test case or select a root that is not treasure box 0.

If we decide to take treasure box 0, i.e. subproblem: (0, true), we cannot take one of its neighbor/children: treasure box 1, i.e. we need to move to subproblem: (1, false).

If we decide not to take treasure box 0, i.e. subproblem: (0, false), we can later also choose not to take one of its neighbor/children: treasure box 1, i.e. we can move to the same subproblem: (1, false) as above. This is a case of overlapping subproblems.

(b.4). (2 points) Analyze the time complexity of your Dynamic Programming solution.
You need to clearly show why it runs in **O(N)** and uses at most **O(N)** table size.

Solution:

*There are 2*N vertices in the subproblem DAG, or O(N), this is the space that we need for our DP table. The time complexity of the DP algorithm, after ensuring each subproblem is only computed once either in top down or bottom up mode, is the number of edges in the subproblem graph, which is 2*(2*N) as each vertex in this subproblem DAG has at most 2 incoming edges. This is O(N). With such solution, N can be as large as 1 ≤ N ≤ 100000 and such solution will still run in a few seconds in today's typical desktop PCs.*

Expected score for this question 3 is (0.75+0.75+1)+(1+0.5+1+0.5) = 5.5 points

Q4. (Greedy Approximation for TREASURE-HUNT) (5 points)

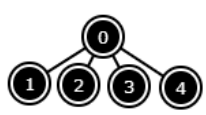
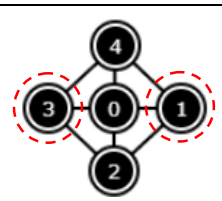
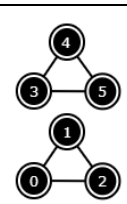
The special case that we have discussed earlier in **Q3 (a)** turns out to be non-existent, i.e. the king's secret note can still contain cases like the sample 4 or 5 shown in **Q2 (a)**.

Fearing that taking the wrong subset of treasure boxes will cause disaster, you hire Professor X from an unknown university Y. After a while, Professor X designs the following algorithm for solving the general case of TREASURE-HUNT. Professor X proudly *claims* that his algorithm is a 1-approximation algorithm, i.e. it is an optimal algorithm.

The pseudo-code of Professor X's algorithm is as follows:

1. Initially all treasure boxes are labeled as "safe";
2. Find **T**, the treasure box with the <u>smallest number</u> of other "safe" treasure boxes connected to it (if tie, choose the treasure box with lower box number), and then take this treasure box T; // **greedy choice**
3. Do NOT take any treasure boxes connected to **T** and mark them as "dangerous";
4. If there is at least one treasure box that is labeled as "safe", repeat step 2;

(a). (1 point) Apply Professor X's algorithm on king's secret note 1, 2, 3, 5 as in **Q2 (a)** and copied below. This time, the solution returned by Professor X's algorithm for king's secret note 4 is already shown: The algorithm first picks treasure box 1, as it has 3 treasure boxes connected to it and it has *the lowest box number*. We mark the neighboring treasure boxes {0, 2, 4} as "dangerous". We are now left with the last safe treasure box 3 and we take it.

Secret Note 1	Secret Note 2	Secret Note 3	Secret Note 4	Secret Note 5
3 2 0 1 1 2	2 1 0 1	5 4 0 1 0 2 0 3 0 4	5 8 0 1 0 2 0 3 0 4 1 2 2 3 3 4 4 1	6 6 0 1 0 2 1 2 3 4 3 5 4 5
				
Ans:__box(es) Take {_____}	Ans:__box(es) Take {_____}	Ans:__box(es) Take {_____}	Ans: 2 box(es) Take {1, 3}	Ans: __ box(es) Take {_____}
Ans: 2 boxes Take {0, 2}	Ans: 1 box(es) Take {0}	Ans: 4 box(es) Take {1, 2, 3, 4}		Ans: 2 box(es) Take {0, 3}

(b). (2 points) Step 2 and 3 of Professor X's pseudo code are two big steps. Please complete the design of Professor X's algorithm by using good data structure(s) so that the algorithm runs in O((N + M) * log N).

Solution:

The idea is to use Priority Queue PQ that orders the vertices in decreasing degrees.

The initialization phase of this PQ is O(N + M) to compute the degrees of each vertex (assuming it is stored in an Adjacency List), then inserting these values to PQ in O(N log N).

Then, after we take a vertex (we flag it as taken) and include it in the Independent Set, we explore its neighbors, flag them as unsafe, and decrease the degree of the affected vertices and update the associated key in PQ (using decreaseKey operation). Notice that we will only process each edge once and each time we do at most O(log N), so we have O(M log N) here.

*In overall: O((N + M) + (N + M) * log N) = O((N + M) * log N).*

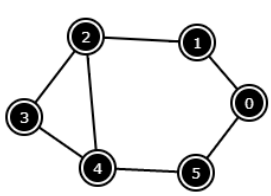
(c). (2 points) Due to your previous conclusion in **Q2 (d)**, it is unlikely that Professor X's polynomial time algorithm is optimal unless P = NP. Show that Professor X's algorithm is **not** an optimal greedy algorithm by supplying one instance of the optimization version TREASURE-HUNT where the number of treasure boxes taken in the optimal solution is at least one more than the number of treasure boxes taken by Professor's X algorithm. Note that to get the points in this question, you need to clearly explain your answer!

Hint: Consider instances with N > 5 treasure boxes.

Solution:

The source of idea is from Wikipedia: Max Independent Set does not have a constant factor approximation algorithm (unless P = NP). But in special graphs, like bounded degree graphs, effective approximation algorithms are known with worse than constant approximation ratios; for instance, a greedy algorithm that forms a maximal independent set by, at each step, choosing the minimum degree vertex in the graph and removing its neighbors, achieves an approximation ratio of (Δ+2)/3 on graphs with maximum degree Δ.

The greedy approximation algorithm, i.e. that Professor X's algorithm surprisingly works for many hand-drawn test cases, including for all given test cases in Q2 (a)/Q4 (a) and finding the answer for this part (c) can be surprisingly difficult (the last CS3230 tutorial 11 has such exercise). One possible answer: Professor X's algorithm will return Independent Set = {0, 2} of size 2 while the optimal one is Independent Set {1, 3, 5} of size 3 (a few other possible drawings given in the appendix).



Expected score for this question 4 is (1)+(1)+(1) = 3 points

Q5. (Solve small instances of TREASURE-HUNT) (5 points)

The special case that we have discussed earlier in **Q3 (a)** turns out to be non-existent, i.e. the king's secret note can still contain cases like the sample 4 or 5 shown in **Q2 (a).** Moreover, Professor X's algorithm discussed in **Q4** turns out to be sub-optimal.

Then you notice a very small footnote in the king's secret note: Hundred years ago (before computers were invented), the king had successfully asked one of his advisers (a human) to compute the answer for this TREASURE-HUNT problem **manually**. You traverse the island one more time and this time you count the number of treasure boxes. You encounter no more than **N** ≤ 60 treasure boxes. Now, design the best possible algorithm (or heuristic) to solve the small instances of TREASURE-HUNT problem.

Hint: Trying all $O(2^N)$ subsets to see if it is an Independent Set (doable in O(M) time) and keeping the largest subset is not possible and will receive 0 point.

Solution:

This is a very hard question with 0, 2, or 5 points answers and will be used to separate the A/A+ students versus the rest of the class.

An exam savvy student should have noticed the keyword "Independent Set" in the hint which tells you the true name of this TREASURE-HUNT problem and probably browse relevant text book chapters related to this NPC problem ☺. Sometime it pays off to read all questions first.

Preliminary marking scheme (adjustable after seeing student's answers):

1. *(0 point) Trying all 2^N subsets is not possible as N can be as big as 60 and 2^{60} is impossible to be done in reasonable time.*
2. *(0 point) We cannot use Dynamic Programming as there are 2^N subsets to be tried and we cannot store them in a table.*
3. *(2 points) However, if we just store some 2^L subsets where L is up to 20, we can speed up <u>part of</u> the (top-down) recursion.*
4. *(2 points) Partial idea of Branch and Bound: We keep track of the best MIS size so far and prune the recursion tree if the current IS size + size of the remaining vertex < best MIS so far. This will help a bit.*
5. *(2 points) We need to immediately reduce the size of available vertices upon deciding to take a certain treasure box in our recursion.*
6. *(5 points) We do idea no 4 above with compressed Adjacency Matrix using bitmask (long long) and heavy bit manipulation operations. Solution is in CP3 page 302-303. Such solution will be a bit hard to analyse though :O.*

Expected score for this question 5 is = 2 points

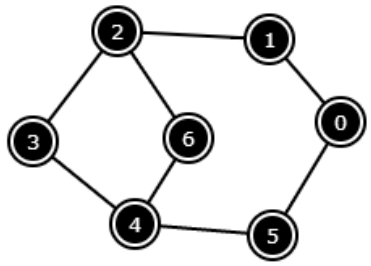
Expected score for the whole paper is = 8+6+5.5+3+2 = 24.5 out of 40 points (~61.25%), i.e. this is a hard paper.

-- End of Paper --

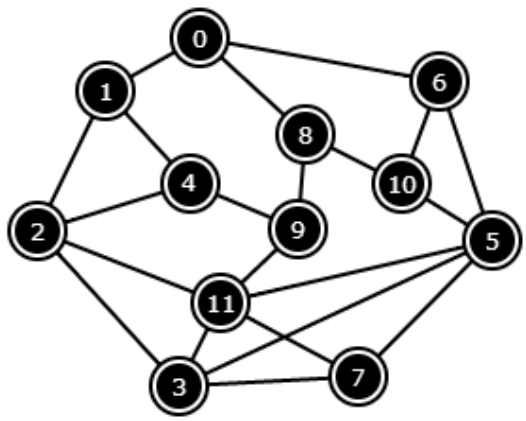
```
// Appendix, code for Q2. b
// code for MIS on tree, students are not expected to write this
// for testing only

#include <bits/stdc++.h>
using namespace std;

#define MAX_N 10000

typedef vector<int> vi;

int memo[MAX_N][2]; // O(2N) = O(N) space
vector<vi> Children;

int MIS(int u, int flag) { // Maximum Independent Set, O(N)
  int ans = 0;
  if (memo[u][flag] != -1)
    return memo[u][flag]; // top down DP
  else if ((int)Children[u].size() == 0) // a leaf has no children
    ans = flag; // 1/0 = taken/not
  else if (flag == 1) { // if u is taken, we must not take its children
    ans = 1;
    for (int j = 0; j < (int)Children[u].size(); j++)
      ans += MIS(Children[u][j], 0);
  }
  else if (flag == 0) { // if u is not taken, take the maximum between
    ans = 0; // taking or not taking its children
    for (int j = 0; j < (int)Children[u].size(); j++)
      ans += max(MIS(Children[u][j], 1), MIS(Children[u][j], 0));
  }
  return memo[u][flag] = ans;
}

int main() {
/*
  // King's secret note 1, ans = 2
  Children.assign(3, vi());
  Children[0].push_back(1);
  Children[1].push_back(2);

  // King's secret note 2, ans = 1
  Children.assign(2, vi());
  Children[0].push_back(1);

  // King's secret note 3, ans = 4
  Children.assign(5, vi());
  Children[0].push_back(1);
```

```
  Children[0].push_back(2);
  Children[0].push_back(3);
  Children[0].push_back(4);

  // King's secret note XX, ans = 5
  Children.assign(8, vi());
  Children[0].push_back(1);
  Children[0].push_back(2);
  Children[0].push_back(3);
  Children[1].push_back(4);
  Children[1].push_back(5);
  Children[3].push_back(6);
  Children[3].push_back(7);
*/

  // King's secret note YY, ans = 6
  Children.assign(10, vi());
  Children[0].push_back(1);
  Children[0].push_back(2);
  Children[0].push_back(3);
  Children[1].push_back(4);
  Children[1].push_back(5);
  Children[3].push_back(6);
  Children[3].push_back(7);
  Children[5].push_back(8);
  Children[5].push_back(9);

  memset(memo, -1, sizeof memo);
  printf("%d\n", max(MIS(0, 0), MIS(0, 1)));

  return 0;
}
```

Appendix: Alternative answers to Q4 (c)



Alternative answer: Professor X's algorithm will return {0, 2, 4} of size 3 whereas the optimal one is {1, 3, 5, 6} of size 4. Note that this graph is a bipartite graph, we can put {2, 0, 4} on the left and {1, 3, 5, 6} on the right.



Another example: Graph with N = 12 and M = 21. MIS = 5, i.e. {0, 2, 7, 9, 10}.
Professor X's algorithm will give MIS = 4, i.e. {0, 3, 4, 10}.

Initial PQ:
{ ~~(3,0)~~, (3,1), (3,4), (3,6), (3,7), (3,8), (3,9), (3,10), (4,2), (4,3), (5,5), (5,11) };

We take vertex 0, flag {1, 6, 8} as dangerous, then update PQ:
{ ~~(1,10)~~, (2,4), (2,9), (3,2), (3,7), (4,3), (4,5), (5,11) };

We take vertex 10, flag {5} as dangerous, then update PQ:
{ ~~(2,4)~~, (2,7), (2,9), (3,2), (3,3), (4,11) };

We take vertex 4, flag {2, 9} as dangerous, then update PQ:
{ ~~(2,3)~~, (2,7), (2,11) };

We take vertex 3, flag {7, 11} as dangerous, PQ is empty, stop.
We have taken MIS = 4 = {0, 10, 4, 3} = {0, 3, 4, 10}